

IE University

School of Science & Technology



Ephemeral Interfaces: Design Patterns for Task-Scoped Generative UI Components

Subtitle of the project

Daniel Kumlin

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Prof. Jose Antonio Garcia Escandón

Abstract

Your abstract goes here

Keywords: Keyword1, Keyword2, Keyword3

1 Introduction

How can ephemeral, task-scoped generative UI components be designed to improve existing applications without disrupting the user experience?

1.1 Literature Review

While LLMs have been primarily used to generate content through text, they are increasingly being used to generate user interfaces (UI). It is a way to move beyond static interfaces by generating structures based on a set of primitives tailored to the users' goals. Prior work suggests that when working with LLMs, users prefer to express themselves through UI instead of a plain chat. This is because the system can present its functionality properly instead of forcing it through text [1, 5, 7]. However, most of the evidence comes from the context of a fully generated UI, which differs from the design challenge of the capstone, which is to augment existing applications where users expect a certain degree of stability and predictability in their tools. Thus, this research motivates me to understand how we can use a set of core primitive UI components in already established workflows instead of replacing the whole state of our modern day interfaces with AI.

The problem with generative UI interfaces is that while they can increase the relevance of the task at hand, they can threaten predictability and user control. In real-world domains such as banking, their rigid interfaces were built on a set of predefined rules that fail to meet the ever-changing needs of their users across services [6]. Approaches that dynamically generate interfaces can address the problem of not tailoring just to the content needed to serve the user, but the actual structure of the interaction over time. Yet, these new systems raise an unresolved question: when generative UI is beneficial, how can it be introduced to users without it corroding the user's mental model of how the application should work? This implies that the design pattern put in place for generative UI must manage the tradeoff between adaptivity and consistency [2].

To actually manage the tradeoff of leveraging generative UI, instead of having the entire interface be generated with AI, we could restrict it to ephemeral, so that task-scoped compo-

nents appear only when needed and disappear when their use case has passed. Past work on ephemeral interfaces argues that the permanence in ubiquitous environments can contribute to cognitive overload as there are too many controls and artifacts present [4]. Conversely, ephemerality can reduce the persistent clutter of artifacts and help with better interactions for the user within a specific time frame and context. While this research was grounded in physical interaction design, it provides principles that can be transferred to digital systems. These ephemeral elements should be discoverable when needed, easy to dismiss, and designed to disappear without affecting the surrounding environment's knowledge. For generative UI, it suggests that ephemeral components could be a natural fit for LLMs as the system could generate the UI contextually in almost real time [2]. However, it must be kept in mind how the component lifecycle should be carefully designed to prevent an unstable user experience.

Ephemeral generative components have been implemented in past research without a full interface replacement, but with a temporary UI as an intermediate layer for exploration and understanding. For agentic programming, research used ephemeral interfaces that sat between the prompt and code generation to improve observability and comprehension of LLM generated code. It helped developers steer their results without needing to inspect the raw code [3]. This shows how ephemeral UI can function as a way to help users with a workflow that lets them steer the outcome and iteratively refine stochastic tasks for exploring alternatives before taking concrete actions. However, this application of ephemeral UI was done in a narrow domain and intent. This leaves the question of how ephemeral patterns can generalise to everyday productivity applications where tasks are diverse. If not implemented properly it could result in a disruption of interruptions, context switching, or inconsistency of familiar workflow.

Taking all existing literature into account, it establishes three relevant and incomplete variables to this capstone. Firstly, fully generative UI experiences can improve the user experience and enjoyment of interacting with generative AI applications compared to only chat-only interaction [1, 5, 7]. Secondly, deployments of dynamic interface generations in specific domains can address the shortcomings of problems in need of rule-based logic where needs are constantly changing and require tailoring [6]. Thirdly, research on ephemerality shows how temporary time-bound interaction artifacts can reduce cognitive overload by decreasing the amount

of permanent UI accumulation for the user. Nonetheless, a clear gap remains at the intersection of limited guidance on how to design *ephemeral, task-scoped generative UI components* that can improve existing applications while preserving a predictable user experience. Prior work does not articulate reproducible design patterns that define when generative components should appear, how their scope should be bounded with an exit that does not disrupt the users' mental model of the application, and how they should preserve a predictable user interface. This capstone addresses that gap by generalising ephemerality into a framework of component patterns to integrate into everyday applications.

2 Methodology

2.1 Research Design

2.2 Conceptual Framework

2.3 Artifact Development

2.4 Evaluation Design

2.4.1 Experimental Design

2.4.2 Participants

2.4.3 Materials and Procedure

2.4.4 Measures

Table 1: Dependent measures and collection methods.

Measure	Type	Collection	Analysis
Task completion rate	Binary	Event logging	McNemar/ χ^2
Task completion time	Continuous	Timestamps	Paired t -test
Interaction count	Discrete	Click counter	Paired t -test
Intrusiveness	Ordinal (1–7)	Likert items	Paired t -test
Preference	Nominal	Forced choice	Binomial test

2.4.5 Data Collection

2.4.6 Analysis Plan

3 Experiments

This is your experimental design text ...

4 Results

This is your results section text

5 Discussion

This is your discussion text

6 Future Work

This is your future work text

7 Conclusion

This is your conclusions

Your supplementary materials are placed here.

References

- [1] Ammar Ahmed and Ali Shariq Imran. “The Role of Large Language Models in UI/UX Design: A Systematic Literature Review”. In: *arXiv preprint arXiv:2507.04469* (2025). URL: <https://arxiv.org/abs/2507.04469>.
- [2] J. Bieniek, M. Rahouti, and D. C. Verma. “Generative AI in Multimodal User Interfaces: Trends, Challenges, and Cross-Platform Adaptability”. In: *arXiv preprint arXiv:2411.10234* (2024). URL: <https://arxiv.org/pdf/2411.10234>.
- [3] Ruijia Cheng et al. “BISCUIT: Scaffolding LLM-Generated Code with Ephemeral UIs in Computational Notebooks”. In: *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 2024. URL: <https://arxiv.org/pdf/2404.07387>.
- [4] Tanja Döring, Axel Sylvester, and Albrecht Schmidt. “Ephemeral User Interfaces: Valuing the Aesthetics of Interface Components That Do Not Last”. In: *ACM Interactions* 20.4 (July 2013), pp. 32–37. URL: <https://interactions.acm.org/archive/view/july-august-2013/ephemeral-user-interfaces>.
- [5] Yaniv Leviathan et al. “Generative UI: LLMs are Effective UI Generators”. In: *arXiv preprint arXiv:2410.11354* (2025). URL: <https://generativeui.github.io/static/pdfs/paper.pdf>.
- [6] Varun Kumar Tambi. “Generative AI Applications in Customizing User Experiences in Banking Apps”. In: *The Research Journal (TRJ)* 6.6 (Nov. 2020), pp. 1–15.
- [7] Fei Tang et al. “A Survey on (M)LLM-Based GUI Agents”. In: *arXiv preprint arXiv:2504.13865* (2025). URL: <https://arxiv.org/abs/2504.13865>.